Module - I

INTRODUCTION:

- ✓ ***Software Engineering is a framework for building software and is an engineering approach to software development.*** Software programs can be developed without S/E principles and methodologies, but they are indispensable if we want to achieve good quality software in a cost-effective manner.
- ✓ Engineering is the branch of science and technology concerned with the design, building, and use of engines, machines, and structures. It is the application of science, tools and methods to find cost effective solutions to simple and complex problems.

SOFTWARE ENGINEERING is defined as a systematic, disciplined, and quantifiable approach for the development, operation and maintenance of software.

The Evolving role of software:

The dual role of Software is as follows:

1. A Product- Information transformer producing, managing, and displaying information.
2. A Vehicle for delivering a product- Control of computer (operating system), the communication ofinformation(networks) and the creation of other programs.

Software:

Software is *Instructions + Data Structures + Documents*

- ✓ *instructions* (computer programs) that when executed provide desired function and performance,
- ✓ *data structures* that enable the programs to adequately manipulate information, and
- ✓ *documents* that describe the operation and use of the programs.

Characteristics of software

- ✓ **Software is developed or engineered**, but it is not manufactured in the classical sense.
- ✓ **Software does not wear out,** but it deteriorates due to change.

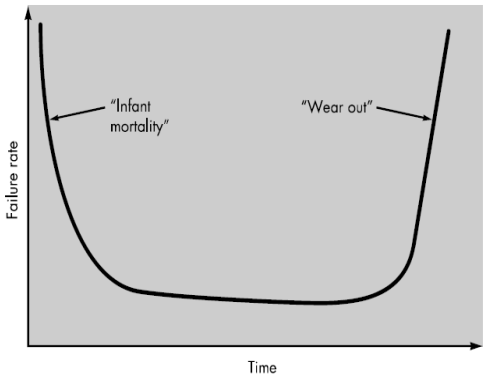


FIGURE 1.1 Failure curve for hardware

The figure of failure curve is also called as bathtub curve indicates that hardware exhibits relatively high failure rates early in its life; defects are corrected and the failures rate drops to a steady state for some period of time. Failure raises again as hardware components suffer from cumulative affects of environmental maladies (hardware wear .out)

VTUSYNC.IN

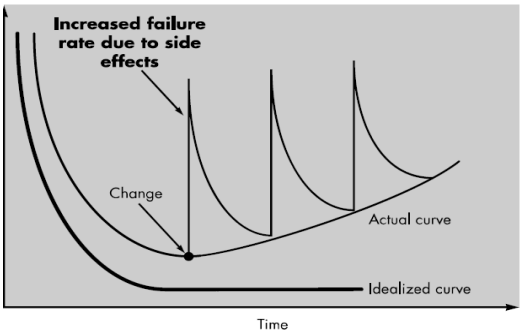


FIGURE 1.2
Idealized and actual failure curves for software

Software is not susceptible to the environment maladies. Failure rate curve for software is as shown in figure. Undiscovered defects will cause high failures rates in the beginning of the program, these failures are corrected and the curve flattens. Spikes in the curve is mainly due to some new defects. But software deteriorates due to change.

- ✓ **Software is custom built** rather than assembling existing components.

Software applications:

- ✓ **System software:** This class of ***software manages and controls the internal operations of a computer system***. It is a group of programs, which is responsible for using computer resources efficiently and effectively. For example, an operating system is a system software, which controls the hardware, manages memory and multitasking functions, and acts as an interface between application programs and the computer.
- ✓ **Real-time software:** This class of **software observes, analyses, and controls real world events as they occur.** Generally, a real-time system guarantees a response to an external event within a specified period of time. An example of real-time software is the software used for weather forecasting that collects and processes parameters like temperature and humidity from the external environment to forecast the weather. Most of the defence organizations all over the world use real-time software to control their military hardware.
- ✓ **Business software:** This class of software is widely used in areas ***where management and control of financial activities is of utmost importance***. The fundamental component of a business system comprises payroll, inventory, and accounting software that permit the user to access relevant data from the database. These activities are usually performed with the help of specialized business software that facilitates efficient framework in business operations and in management decisions.
- ✓ **Engineering and scientific software:** This class of software has emerged as a powerful tool in the ***research and development of next generation technology***. Applications such as the study of celestial bodies, under-surface activities, and programming of an orbital path for space shuttles are heavily dependent on engineering and scientific software. This software is designed to perform precise calculations on complex numerical data that are obtained during real-time environment.
- ✓ **Artificial intelligence (AI) software:** This class of software is used where the problem-solving technique is non-algorithmic in nature. The solutions of such problems are generally non-agreeable to computation or straightforward analysis. Instead, these problems require specific problem- solving strategies that include expert system, pattern recognition, and game- playing techniques. In addition, they involve different kinds of search techniques which include the use of heuristics. The role of artificial intelligence software is to add certain degrees of intelligence to the mechanical hardware in order to get the desired work done in an agile manner.

vtusync.in

- ✓ **Embedded Software :** This type of software is embedded into the hardware normally in ***the Read-Only Memory (ROM) as a part of a large system*** and is used to support certain functionality under control conditions.
  Examples are software used in instrumentation and control applications like washing machines, satellites, microwaves, etc.
- ✓ **Web-based software:** This class of ***software acts as an interface between the user and the Internet. Data on the Internet is in the form of text, audio, or video format, linked with hyperlinks.*** Web browser is a software that retrieves web pages from the Internet. The software incorporates executable instructions written in special scripting languages such as CGI or ASP. Apart from providing navigation on the Web, this software also supports additional features that are useful while surfing the Internet.
- ✓ **Personal computer (PC) software:** This class of software is used for ***both official and personal use.*** The personal computer software market has grown over in the last two decades from normal text editor to word processor and from simple paintbrush to advanced image-editing software. This software is used predominantly in almost every field, whether it is database management system, financial accounting package, or multimedia-based software. It has emerged as a versatile tool for routine applications.

Legacy Software

- ✓ Legacy software is an older program that are developed decades ago. The quality of legacy software is poor because it has inextensible design, convoluted code, poor and nonexistent documentation, test casesand results that are not achieved.

As time passes legacy systems evolve due to following reasons:

- The software must be adapted to meet the needs of new computing environment or technology.
- The software must be enhanced to implement new business requirements.
- The software must be extended to make it interoperable with more modern systems or database
- The software must be rearchitected to make it viable within a network environment.

Unique nature of Web Apps

The following attributes are encountered in the vast majority of WebApps.

- ✓ **Network intensiveness.** A WebApp resides on a network and must serve the needs of a diverse community of clients. The network may enable worldwide access and communication (i.e., the Internet) or more limited access and communication (e.g., a corporate Intranet).
- ✓ **Concurrency:** A large number of users may access the WebApp at one time. In many cases, the patterns of usage among end users will vary greatly.
- ✓ **Unpredictable load:** The number of users of the WebApp may vary by orders of magnitude from day to day. One hundred users may show up on Monday; 10,000 may use the system on Thursday.
- ✓ **Performance:** If a WebApp user must wait too long (for access, for server side processing, for client-side formatting and display), he or she may decide to go elsewhere.

VTUSYNC.IN

- ✓ **Availability:** Although expectation of 100 percent availability is unreasonable, users of popular WebApps often demand access on a 24/7/365 basis. Users in Australia or Asia might demand access during times when traditional domestic software applications in North America might be taken off-line for maintenance.
- ✓ **Data driven:** The primary function of many WebApps is to use hypermedia to present text, graphics, audio, and video content to the end user. In addition, WebApps are commonly used to access information that exists on databases that are not an integral part of the Web-based environment (e.g., e-commerce or financial applications).
- ✓ **Content sensitive:** The quality and aesthetic nature of content remains an important determinant of the quality of a WebApp.
- ✓ **Continuous evolution:** Unlike conventional application software that evolves over a series of planned, chronologically spaced releases, Web applications evolve continuously. It is not unusual for some WebApps (specifically, their content) to be updated on a minute-by-minute schedule or for content to be independently computed for each request.
- ✓ **Immediacy:** Although immediacy the compelling need to get software to market quickly is a characteristic of many application domains, WebApps often exhibit a time-to-market that can be a matter of a few days or weeks.
- ✓ **Security:** Because WebApps are available via network access, it is difficult, if not impossible, to limit the population of end users who may access the application. In order to protect sensitive content and provide secure modes of data transmission, strong security measures must be implemented throughout the infrastructure that supports a WebApp and within the application itself.
- ✓ **Aesthetics:** An undeniable part of the appeal of a WebApp is its look and feel. When an application has been designed to market or sell products or ideas, aesthetics may have as much to do with success as technical design.

Software Engineering:

- ❖ **To build software that is ready to meet the current challenges, recognize a few simple realities:**
- ❖ **Understand the problem before you build a solution**
- ❖ **Design is a pivotal software engineering activity.**
- ❖ **Both quality and maintainability are an outgrowth of good design**

Software Engineering - A Layered Technology

figure 1.3 Software engineering layer



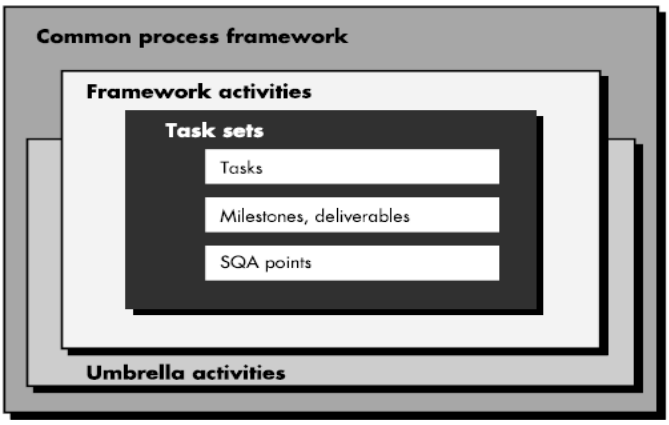
Quality focus - Bedrock that supports Software Engineering.
Process - Foundation for software Engineering**. Is the glue that holds technology layers together and enables rational and timely development of software.** Process defines framework for a set of key process areas, which form the basis for management control of software projects.

- ❖ **Methods** - Provide technical How-to's for building software. Methods include requirement analysis, design, program construction, testing and support.
- ❖ **Tools** - Provide semi-automated and automated support to methods and process.

VTUSYNC.IN

A Process Framework

- ✓ Establishes the foundation for a complete software process.
- ✓ Identifies a number of framework activities applicable to all software projects.
- ✓ Include a set of umbrella activities that are applicable across the entire software process.



Process framework determines the processes which are essential for completing a complex software project. This framework identifies certain activities, known as **framework activities**, which are applicable to all software projects regardless of their type and complexity. Some of the framework activities are listed below.

- **Communication:** It involves communication with the user so that the requirements are easily understood.
- **Planning:** It establishes a plan for accomplishing the project. It describes the schedule for the project, the technical tasks involved, expected risks, and the required resources.

✓ **Modeling:** It encompasses creation of models, which allow the developer and the user to understand software requirements and the designs to achieve those requirements.

✓ **Construction:** It combines generation of code with testing to uncover errors in the code.

✓ **Deployment:** It implies that the final product (software) is delivered to the user. The user evaluates the delivered product and provides feedback based on the evaluation.

In addition to framework activities, process framework also comprises a set of activities known as **umbrella activities**, which are used throughout the software process. These activities are listed below.

✓ **Software project tracking and control:** It monitors the actual process so that management can take necessary steps if the software project deviates from the plans laid down. It involves tracking procedures and reviews to check whether the software project is according to user's requirements. A documented plan is used as a basis for tracking the software activities and revising the plans.

✓ **Formal technical reviews:** assesses software engineering work products to uncover and remove errors before they are propagated to the next activity. It assesses the code, products, and documents of software engineering practices to detect and remove errors.

✓ **Software quality assurance: D**efines and conducts the activities required to ensure software quality. It assures that the software is according to the requirements. In addition, it is designed to evaluate the processes of developing and maintaining quality of the software.

✓ **Reusability management:** It determines the criteria for products' reuse and establishes mechanisms to achieve reusable components.

✓ **Software configuration management:** It manages the changes made in the software processes of the products throughout the life cycle of the software project. It controls changes made to the configuration and maintains the integrity in the software development process.

VTUSYNC.IN

- ✓ **Risk management: A**ssesses risks that may affect the outcome of the project or the quality of the product. It identifies, analyses, evaluates, and eliminates the possibility of unfavourable deviations from expected results, by following a systematic activity and then develops strategies to manage them.
- ✓ **Measurement**: Defines and collects process, project, and product measures that assist the team in delivering software that meets stakeholders' needs; can be used in conjunction with all other framework and umbrella activities.

Work product preparation and production—encompasses the activities required to create work products such as models, documents, logs, forms, and lists.

SOFTWARE ENGINEERING PRACTICE

- ✓ Generic framework activities—**communication, planning, modeling, construction,** and **deployment** and umbrella activities establish a skeleton architecture for software engineering work.

The Essence of Practice : Essence of software engineering practice:

1. Understand the problem (communication and analysis).
2. Plan a solution (modeling and software design)
3. Carry out the plan (code generation).
4. Examine the result for accuracy (testing and quality assurance).

1. **Understand the problem**. It's sometimes difficult to admit, but most of us suffer from hubris when we're presented with a problem.
   - Who has a stake in the solution to the problem? That is, who are the stakeholders?
   - What are the unknowns? What data, functions, and features are required to properly solve the problem?
   - Can the problem be compartmentalized? Is it possible to represent smaller problems that may be easier to understand?
   - Can the problem be represented graphically? Can an analysis model be created?
2. **Plan the solution.** Now you understand the problem (or so you think) and you can't wait to begin coding.
   - Have you seen similar problems before? Are there patterns that are recognizable in a potential solution? Is there existing software that implements the data, functions, and features that are required?
   - Has a similar problem been solved? If so, are elements of the solution reusable?
   - Can subproblems be defined? If so, are solutions readily apparent for the subproblems?
   - Can you represent a solution in a manner that leads to effective implementation? Can a design model be created?
3. **Carry out the plan**. The design you've created serves as a road map for the system you want to build. There may be unexpected detours, and it's possible that you'll discover an even better route as you go, but the "plan" will allow you to proceed without getting lost.
   - Does the solution conform to the plan? Is source code traceable to the design model?

VTUSYNC.IN

- Is each component part of the solution provably correct? Have the design and code been reviewed, or better, have correctness proofs been applied to the algorithm?

4. **Examine the result**. You can't be sure that your solution is perfect, but you can be sure that you've designed a sufficient number of tests to uncover as many errors as possible.
   - Is it possible to test each component part of the solution? Has a reasonable testing strategy been implemented?
   - Does the solution produce results that conform to the data, functions, and features that are required? Has the software been validated against all stakeholder requirements? It shouldn't surprise you that much of this approach is common sense. In fact, it's reasonable to state that a commonsense approach to software engineering will never lead you astray.

General Principles

The dictionary defines the word principle as "an important underlying law or assumption required in a system of thought." David Hooker [Hoo96] has proposed seven principles that focus on software engineering practice. They are reproduced in the following paragraphs:

1) **The First Principle: The Reason It All Exists**

A software system exists for one reason: to provide value to its users. All decisions should be made with this in mind. Before specifying a system requirement, before noting a piece of system functionality, before determining the hardware platforms or development processes, ask yourself questions such as: "Does this add real value to the system?" If the answer is "no," don't do it. All other principles support this one.

2) **The Second Principle: KISS (Keep It Simple, Stupid!)**

Software design is not a haphazard process. There are many factors to consider in any design effort. All design should be as simple as possible, but no simpler. This facilitates having a more easily understood and easily maintained system. This is not to say that features, even internal features, should be discarded in the name of simplicity. Indeed, the more elegant designs are usually the more simple ones. Simple also does not mean "quick and dirty." In fact, it often takes a lot of thought and work over multiple iterations to simplify. The payoff is software that is more maintainable and less error-prone.

3) **The Third Principle: Maintain the Vision**

A clear vision is essential to the success of a software project. Without one, a project almost unfailingly ends up being "of two [or more] minds" about itself. Without conceptual integrity, a system threatens to become a patchwork of incompatible designs, held together by the wrong kind of screws. Compromising the architectural vision of a software system weakens and will eventually break even the well-designed systems. Having an empowered architect who can hold the vision and enforce compliance helps ensure a very successful software project.

4) **The Fourth Principle: What You Produce, Others Will Consume**

Seldom is an industrial-strength software system constructed and used in a vacuum. In some way or other, someone else will use, maintain, document, or otherwise depend on being able to understand your system. So, always specify, design, and implement knowing someone else will have to understand what you are doing. The audience for

VTUSYNC.IN

any product of software development is potentially large. Specify with an eye to the users. Design, keeping the implementers in mind. Code with concern for those that must maintain and extend the system. Someone may have to debug the code you write, and that makes them a user of your code.

5) **The Fifth Principle: Be Open to the Future**

A system with a long lifetime has more value. In today's computing environments, where specifications change on a moment's notice and hardware platforms are obsolete just a few months old, software lifetimes are typically measured in months instead of years. However, true "industrial-strength" software systems must endure far longer. To do this successfully, these systems must be ready to adapt to these and other changes. Systems that do this successfully are those that have been designed this way from the start. Never design yourself into a corner. Always ask "what if," and prepare for all possible answers by creating systems that solve the general problem, not just the specific one. This could very possibly lead to the reuse of an entire system.

6) **The Sixth Principle: Plan Ahead for Reuse**

Reuse saves time and effort. Achieving a high level of reuse is arguably the hardest goal to accomplish in developing a software system. The reuse of code and designs has been proclaimed as a major benefit of using object-oriented technologies. However, the return on this investment is not automatic. To leverage the reuse possibilities that object-oriented [or conventional] programming provides requires forethought and planning. There are many techniques to realize reuse at every level of the system development process. . . . Planning ahead for reuse reduces the cost and increases the value of both the reusable components and the systems into which they are incorporated.

7) **The Seventh principle: Think!**

This last principle is probably the most overlooked. Placing clear, complete thought before action almost always produces better results. When you think about something, you are more likely to do it right. You also gain knowledge about how to do it right again.

SOFTWARE MYTHS

Software myths—erroneous beliefs about software and the process that is used to build it can be traced to the earliest days of computing. Myths have a number of attributes that make them insidious. For instance, they appear to be reasonable statements of fact (sometimes containing elements of truth), they have an intuitive feel, and they are often promulgated by experienced practitioners who "know the score."

a. **Management myths.** Managers with software responsibility, like managers in most disciplines, are often under pressure to maintain budgets, keep schedules from slipping, and improve quality. Like a drowning person who grasps at a straw, a software manager often grasps at belief in a software myth, if that belief will lessen the pressure (even temporarily).

- ✓ **Myth:** We already have a book that's full of standards and procedures for building software. Won't that provide my people with everything they need to know?
- ✓ **Reality:** The book of standards may very well exist, but is it used? Are software

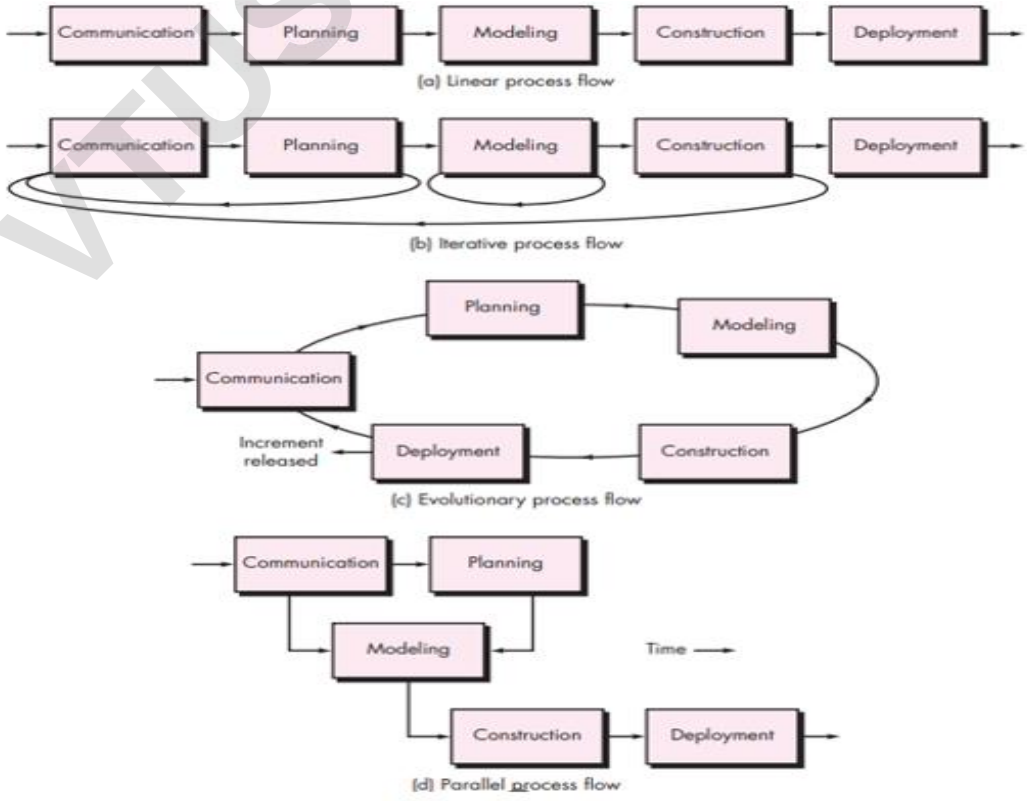
vtusync.in

practitioners aware of its existence? Does it reflect modern software engineering practice? Is it complete? Is it adaptable? Is it streamlined to improve time-to-delivery while still maintaining a focus on quality? In many cases, the answer to all these questions is"no."

- ✓ **Myth:** If we get behind schedule, we can add more programmers and catch up (sometimes called the "Mongolian horde" concept).
- ✓ **Reality:** Software development is not a mechanistic process like statement may seem counterintuitive. However, as new people are added, people who were working must spend time educating the newcomers, thereby reducing the
- ✓ **Myth:** If I decide to outsource the software project to a third party, I can just relax and let that firm build it.
- ✓ **Reality:** If an organization does not understand how to manage and control software projects internally, it will invariably struggle when it outsources software projects.

PROCESS MODEL

- ✓ **A linear process flow** executes each of the five framework activities in sequence, beginning with communication and culminating with deployment (Figure a).
- ✓ **An iterative process** flow repeats one or more of the activities before proceeding to the next (Figure b).



VTUSYNC.IN

- ✓ **An evolutionary process** flow executes the activities in a "circular" manner. Each circuit through the five activities leads to a more complete version of the software (Figure c).
- ✓ **A parallel process flow** (Figure d) executes one or more activities in parallel with other activities (e.g., modeling for one aspect of the software might be executed in parallel with construction of another aspect of the software).

Defining a Framework Activity

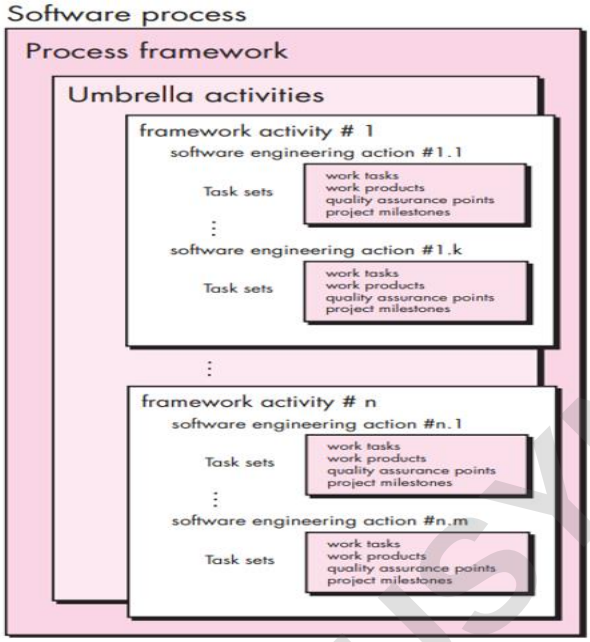


figure 2.1 A software process framework

✓ A software team would need significantly more information before it could properly execute any one of these activities as part of the software process.

✓ For a small software project requested by one person (at a remote location) with simple, straightforward requirements, the **communication activity** might encompass little more than a phone call with the appropriate stakeholder. Therefore, the only necessary action is phone conversation, and the work tasks (the task set) that this action encompasses are:

1. Make contact with stakeholder via telephone.
2. Discuss requirements and take notes.
3. Organize notes into a brief written statement of requirements.
4. E-mail to stakeholder for review and approval.

✓ If the project was considerably more complex with many stakeholders, each with a different set of (sometime conflicting) requirements, the communication activity might have six distinct actions : inception, elicitation, elaboration, negotiation, specification, and validation.

Identifying a Task Set

✓ Referring again to Figure 2.1, each software engineering action (e.g., elicitation, an action associated with the communication activity) can be represented by a number of different task sets—each a collection of software engineering work tasks, related work products, quality assurance points, and project milestones. You should choose a task set that best accommodates the needs of the project and the characteristics of your team. This implies that a software engineering action can be adapted to the specific needs of the software project and the characteristics of the project team.

Process Patterns

✓ A *process pattern* describes a process-related problem that is encountered during software engineering work, identifies the environment in which the problem has been

VTUSYNC.IN

encountered, and suggests one or more proven solutions to the problem. Stated in more general terms, a process pattern provides you with a template —a consistent method for describing problem solutions within the context of the software process.

Patterns can be defined at any level of abstraction.

- ✓ a pattern might be used to *describe a problem (and solution) associated with a complete process model* (e.g., prototyping).
- ✓ patterns can be used to *describe a problem (and solution) associated with a framework activity* (e.g., **planning**) or an action within a framework activity (e.g., project estimating).

Ambler has proposed a template for describing a process pattern:

- ✓ **Pattern Name.** The pattern is given a meaningful name describing it within the context of the software process (e.g., **Technical Reviews**).
- ✓ **Forces.** The environment in which the pattern is encountered and the issues that make the problem visible and may affect its solution.

Type. The pattern type is specified. Ambler suggests three types:

1. *Stage pattern*—defines a problem associated with a framework activity for the process. Since a framework activity encompasses multiple actions and work tasks, a stage pattern incorporates multiple task patterns (see the following) that are relevant to the stage (framework activity). An example of a

stage pattern might be **Establishing Communication.** This pattern would incorporate the task pattern **Requirements Gathering** and others.

2. *Task pattern*—defines a problem associated with a software engineering action or work task and relevant to successful software engineering practice (e.g., **Requirements Gathering** is a task pattern).

3. *Phase pattern*—define the sequence of framework activities that occurs within the process, even when the overall flow of activities is iterative in nature. An example of a phase pattern might be **Spiral Model** or **Prototyping.**

Initial context. Describes the conditions under which the pattern applies. Prior to the initiation of the pattern:

i. What organizational or team-related activities have already occurred?
ii. What is the entry state for the process?
iii. What software engineering information or project information already exists?

For example, the **Planning** pattern (a stage pattern) requires that

i. customers and software engineers have established a collaborative communication;
ii. successful completion of a number of task patterns [specified] for the **Communication** pattern has occurred; and
iii. the project scope, basic business requirements, and project constraints are known.

Problem. The specific problem to be solved by the pattern.

VTUSYNC.IN

Solution. Describes how to implement the pattern successfully. This section describes how the initial state of the process (that exists before the pattern is implemented) is modified as a consequence of the initiation of the pattern. It also describes how software engineering information or project information that is available before the initiation of the pattern is transformed as a consequence of the successful execution of the pattern.

Resulting Context. Describes the conditions that will result once the pattern has been successfully implemented. Upon completion of the pattern:

I. What organizational or team-related activities must have occurred?
II. What is the exit state for the process?
III. What software engineering information or project information has been developed?

- ✓ **Related Patterns.** Provide a list of all process patterns that are directly related to this one. This may be represented as a hierarchy or in some other diagrammatic form. For example, the stage pattern **Communication** encompasses the task patterns: **Project Team, Collaborative Guidelines, Scope Isolation, Requirements Gathering, Constraint Description,** and **Scenario Creation.**
- ✓ **Known Uses and Examples.** Indicate the specific instances in which the pattern is applicable. For example, **Communication** is mandatory at the beginning of every software project, is recommended throughout the software project, and is mandatory once the deployment activity is under way.
- ✓ Process patterns provide an effective mechanism for addressing problems associated with any software process. The patterns enable you to develop a hierarchical process description that begins at a high level of abstraction (a phase pattern). The description is then refined into a set of stage patterns that describe framework activities and are further refined in a hierarchical fashion into more detailed task patterns for each stage pattern. Once process patterns have been developed, they can be reused for the definition of process variants—that is, a customized process model can be defined by a software team using the patterns as building blocks for the process model.

INFO



An Example Process Pattern

The following abbreviated process pattern describes an approach that may be applicable when stakeholders have a general idea of what must be done but are unsure of specific software requirements.

Pattern name. **RequirementsUnclear**

Intent. This pattern describes an approach for building a model (a prototype) that can be assessed iteratively by stakeholders in an effort to identify or solidify software requirements.

Type. Phase pattern.

Initial context. The following conditions must be met prior to the initiation of this pattern: (1) stakeholders have been identified; (2) a mode of communication between stakeholders and the software team has been established; (3) the overriding software problem to be solved has been identified by stakeholders; (4) an initial understanding of project scope, basic business requirements, and project constraints has been developed.

Problem. Requirements are hazy or nonexistent, yet there is clear recognition that there is a problem to be solved, and the problem must be addressed with a software solution. Stakeholders are unsure of what they want; that is, they cannot describe software requirements in any detail.

Solution. A description of the prototyping process would be presented here and is described later in Section 2.3.3.

Resulting context. A software prototype that identifies basic requirements (e.g., modes of interaction, computational features, processing functions) is approved by stakeholders. Following this, (1) the prototype may evolve through a series of increments to become the production software or (2) the prototype may be discarded and the production software built using some other process pattern.

Related patterns. The following patterns are related to this pattern: **CustomerCommunication, IterativeDesign, IterativeDevelopment, CustomerAssessment, RequirementExtraction.**

Known uses and examples. Prototyping is recommended when requirements are uncertain.

VTUSYNC.IN

Process Assessment and Improvement

A number of different approaches to software process assessment and improvement have been proposed over the past few decades:

Standard CMMI Assessment Method for Process Improvement (SCAMPI)—provides a five-step process assessment model that incorporates five phases: initiating, diagnosing, establishing, acting, and learning. The SCAMPI method uses the SEI CMMI as the basis for assessment [SEI00].

CMM-Based Appraisal for Internal Process Improvement (CBA IPI)—provides a diagnostic technique for assessing the relative maturity of a software organization; uses the SEI CMM as the basis for the assessment.

PROCESS MODELS

PRESCRIPTIVE PROCESS MODELS

Prescriptive process models were initially designed to bring structure to software development, but the nature of software work and its products often resides in a state between order and chaos known as "the edge of chaos." This state allows for creativity and self-organization while maintaining some level of structure necessary for change. While traditional models prioritize order, they may not suit the dynamic nature of software development. However, abandoning structure entirely could lead to coordination and coherence issues. Finding the right balance is crucial, and various alternatives exist for software engineers to explore. These models prescribe a set of process elements and a process flow, each emphasizing different activities and approaches to achieving project consistency.

VTUSYNC.IN

The Waterfall Model

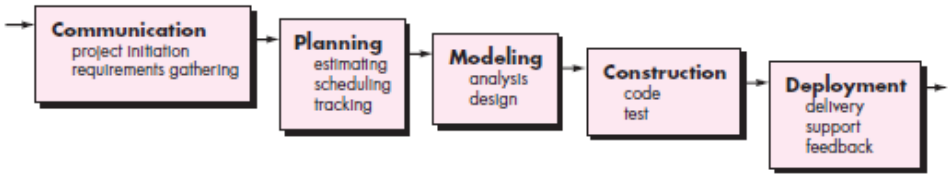
There are instances when the needs for a problem are clear—when workflows from **communication to deployment** in a somewhat linear manner.

Context: Used when requirements are reasonably well understood.

Advantage: It can serve as a useful process model in situations where requirements are fixed and work is to proceed to complete in a linear manner.

The *waterfall model,* sometimes called the *classic life cycle*, suggests a systematic, sequential approach to software development that begins with customer specification of requirements and progresses through planning, modeling, construction, and deployment, culminating in ongoing support of the completed software (Figure 2.3).

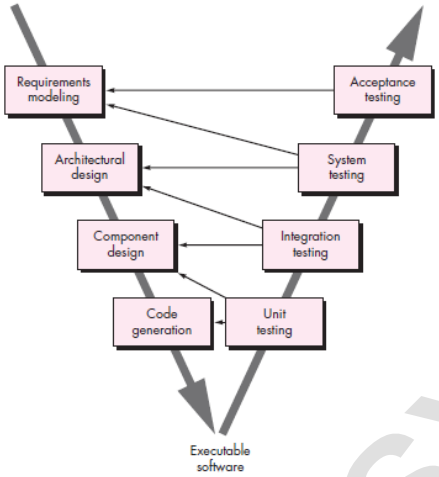
FIGURE 2.3 The waterfall model



The **problems** that are sometimes encountered when the waterfall model is applied are:

1. Real projects rarely follow the sequential flow that the model proposes. Although the linear model can accommodate iteration, it does so indirectly. As a result, changes can cause confusion as the project team proceeds.
2. It is often difficult for the customer to state all requirements explicitly. The waterfall model requires this and has difficulty accommodating the natural uncertainty that exist at the beginning of many projects.
3. The customer must have patience. A working version of the programs will not be available until late in the project time-span. If a major blunder is undetected then it can be disastrous until the program is reviewed.

figure 2.4 The V-model



A variation in the representation of the waterfall model is called the *V-model*. Represented in Figure 2.4, the V-model depicts the relationship of quality assurance actions to the actions associated with communication, modeling, and early construction activities. As a software team moves down the left side of the V, basic problem requirements are refined into progressively more detailed and technical representations of the problem and its solution. Once code has been generated, the team moves up the right side of the V, essentially performing a series of tests (quality assurance actions) that validate each of the models created as the team moved down the left side.

In reality, there is no fundamental difference between the classic life cycle and the V-model. The V-model provides a way of *visualizing how verification and validation* actions are applied to earlier engineering work. The waterfall model is the oldest paradigm for software engineering. However, criticism of this process model has caused even ardent supporters to question its efficacy.

Today, software work is fast-paced and subject to a never-ending stream of changes (to features, functions, and information content). The waterfall model is often *inappropriate* for such work. However, it can serve as a useful process model in situations where *requirements are fixed and work is to proceed to completion in a linear manner.*

Incremental Process Models

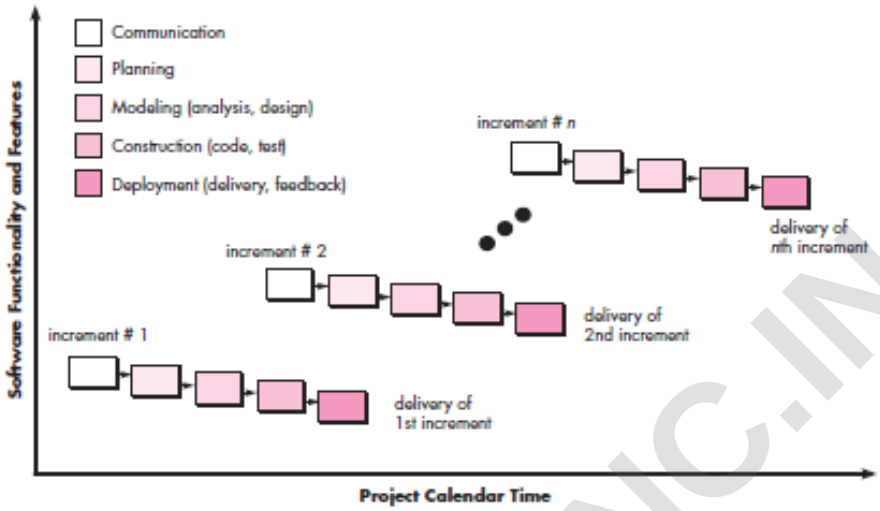
Context:

- ✓ Incremental development is particularly useful when staffing is unavailable for a complete implementation by the business deadline that has been established for the project. Early increments can be implemented with fewer people. If the core product is well received, additional staff can be added to implement the next increment. The *incremental* model combines elements of linear and parallel process flows. Referring to Figure 2.5, the incremental model applies linear sequences in a staggered fashion as calendar time progresses. Each linear sequence produces deliverable **"increments"** of the software.

VTUSYNC.IN

✓ When an incremental model is used, the first increment is often a ***core product***. That is, basic requirements are addressed but many supplementary features (some known, others unknown) remain undelivered. The core product is used by the customer (or undergoes detailed evaluation).

figure 2.5 The incremental model



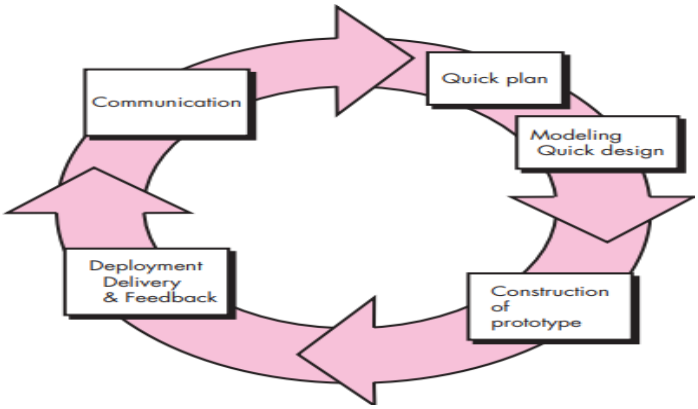
The incremental process model focuses on the delivery of an operational product with each increment. Early increments are stripped-down versions of the final product, but they do provide capability that serves the user and also provide a platform for evaluation by the user.

Incremental development is particularly useful when:

•staffing is unavailable for a complete implementation by the business deadline that has been established for the project. Early increments can be implemented with fewer people. If the core product is well received, then additional staff (if required) can be added to implement the next increment. increments can be planned to manage technical risks.

Evolutionary Process Models

✓ Software, like all complex systems, evolves over a period of time. In these situations, the process model has to be explicitly designed to accommodate a product that evolves over time. Evolutionary models are iterative. They are characterized in a manner that enables you to develop increasingly more complete versions of the software. Two common evolutionary process models.



Prototyping: Prototyping is most frequently employed as a technique that may be applied in conjunction with any of the process models mentioned earlier, however it can also be used as a stand-alone process model. Regardless of the manner in which it is applied, the prototyping paradigm assists you and other stakeholders to *better understand what is to be built when requirements are fuzzy*.

FIGURE 2.6 The prototyping paradigm

VTUSYNC.IN

The first step in the prototype paradigm (Figure is **communication.** Meet with other stakeholders to define the overall objectives for the software, identify whatever requirements are known, and outline areas where further definition is mandatory. A prototyping iteration is **planned quickly**, **and modeling** (in the form of a "quick design") occurs. A quick design focuses on a representation of those aspects of the software that will be visible to end users (e.g., human interface layout or output display formats). The quick design leads to the **construction of a prototype**. The prototype is **deployed** and evaluated by stakeholders, who provide **feedback** that is used to further refine requirements.

Iteration occurs as the prototype is tuned to satisfy the needs of various stakeholders, while at the same time enabling you to better understand what needs to be done. Ideally, the prototype serves as a mechanism for identifying software requirements.

The prototype paradigm is popular with software engineers as well as stakeholders.

Developers can start working on anything right away, while users receive a sense of the real syst em. However, the following factors make prototyping potentially **problematic**:

1. Stakeholders see what appears to be a working version of the software, unaware that the prototype is held together haphazardly, unaware that in the rush to get it working you haven't considered overall software quality or long-term maintainability. When informed that the product must be rebuilt so that high levels of quality can be maintained, stakeholders cry foul and demand that "a few fixes" be applied to make the prototype a working product. Too often, software development management relents.

2. As a software engineer, you often make implementation compromises in order to get a prototype working quickly. An inappropriate operating system or programming language may be used simply because it is available and known; an inefficient algorithm may be implemented simply to demonstrate capability. After a time, you may become comfortable with these choices and forget all the reasons why they were inappropriate. The less-than-ideal choice has now become an integral part of the system

.The Spiral Model.

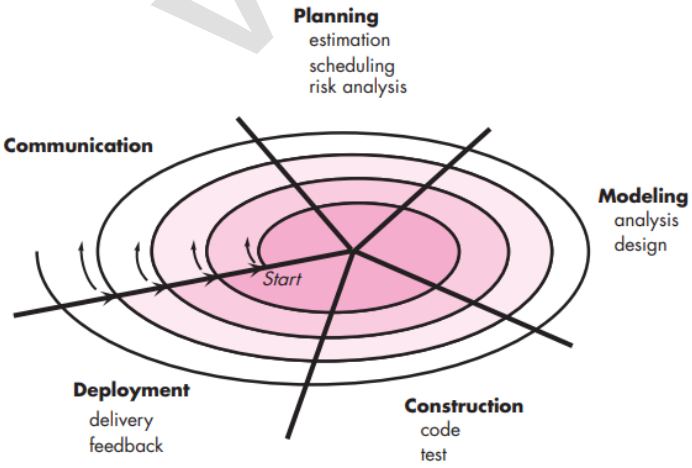


FIGURE 2.7 A typical spiral mode

The spiral model, which was first put forth by Barry Boehm [Boe88], is an evolutionary software process model that combines the waterfall model's regulated and methodical features with the iterative nature of prototyping. It offers the possibility of quickly developing progressively more comprehensive software versions. The model is described by Boehm [Boe01a] as follows: The spiral development model is a **risk-driven process** model generator that is used to guide multi-stakeholder concurrent engineering of software intensive systems.

VTUSYNC.IN

It **has two main distinguishing features**.

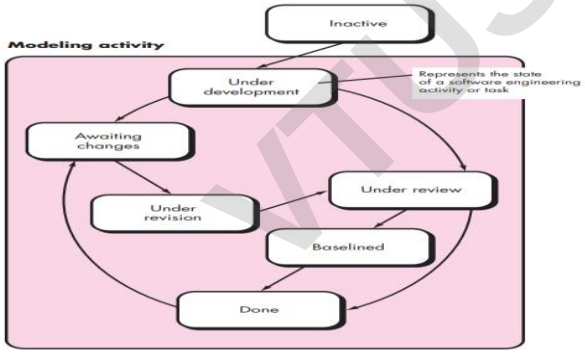
1. **cyclic approach** for incrementally growing a system's degree of definition and implementation while decreasing its degree of risk.
2. Set of **anchor point milestones** for ensuring stakeholder commitment to feasible and mutually satisfactory system solutions. Using the spiral model, software is developed in a series of evolutionary releases. During early iterations, the release might be a model or prototype. During later iterations, increasingly more complete versions of the engineered system are produced.

- ✓ A spiral model is divided into a set of framework activities defined by the software engineering team. Each of the framework activities represent one segment of the spiral path illustrated in Figure. As this evolutionary process begins, the software team performs activities that are implied by a circuit around the spiral in a clockwise direction, beginning at the center.
- ✓ Risk is considered as each revolution is made. *Anchor point milestones* a combination of work products and conditions that are attained along the path of the spiral are noted for each evolutionary pass.

Draw Backs:

The spiral model is not a panacea. It may be difficult to convince customers that the evolutionary approach is controllable. It demands considerable risk assessment expertise and relies on this expertise for success. If a major risk is not uncovered and managed, problems will undoubtedly occur.

THE CONCURRENT DEVELOPMENT MODEL



- The concurrent development model also called concurrent engineering
- Constitutes a series of framework activities, software engineering action, tasks and their associated states
- All activities exist concurrently but reside in different states
- Applicable to all types of software development
- Event generated at one point in the process trigger transitions among the states.

FIGURE 2.8 One element of the concurrent process mode

SPECIALIZED PROCESS MODELS

These models tend to be applied when a specialized or narrowly defined software engineering approach is chosen.

Component-Based Development:

- ✓ Commercial off-the-shelf (COTS) software components, developed by vendors who offer them as products, provide targeted functionality with well-defined interfaces that enable the component to be integrated into the software that is to be built. The component-based development model incorporates many of the characteristics of the spiral model. It is evolutionary in nature, demanding an iterative approach to the

VTUSYNC.IN

creation of software. However, the component-based development model constructs applications from prepackaged software components.

- ✓ Modeling and construction activities begin with the identification of candidate components. These components can be designed as either conventional software modules or object-oriented classes or packages of classes. Regardless of the technology that is used to create the components, the component-based development model incorporates the following steps

1. Available component-based products are researched and evaluated for the application domain in question.
2. Component integration issues are considered.
3. A software architecture is designed to accommodate the components.
4. Components are integrated into the architecture.
5. Comprehensive testing is conducted to ensure proper functionality.

The component-based development model leads to software reuse, and reusability provides software engineers with a number of measurable benefits.

The Formal Methods Model:

- ✓ The formal methods model encompasses a set of activities that leads to formal mathematical specification of computer software. Formal methods enable you to specify, develop, and verify a computer-based system by applying a rigorous, mathematical notation. A variation on this approach, called ***cleanroom software engineering***, is currently applied by some software development organizations.
- ✓ When formal methods are used during design, they serve as a basis for program verification and therefore enable you to discover and correct errors that might otherwise go undetected.
- ✓ Although not a mainstream approach, the formal methods model offers the promise of defect-free software. Yet, concern about its applicability in a business environment has been voiced:
    - The development of formal models is currently quite time consuming and expensive.
    - Because few software developers have the necessary background to apply formal methods, extensive training is required.
    - It is difficult to use the models as a communication mechanism for technically unsophisticated customers.

Aspect-Oriented Software Development:

- ✓ Regardless of the software process that is chosen, the builders of complex software invariably implement a set of localized features, functions, and information content. These localized software characteristics are modeled as components (e.g., object oriented classes) and then constructed within the context of a system architecture.
- ✓ As modern computer-based systems become more sophisticated (and complex), Other concerns affect functions (e.g., the application of business rules), while others are systemic (e.g., task synchronization or memory management).

VTUSYNC.IN

- ✓ When concerns cut across multiple system functions, features, and information, they are often referred to as crosscutting concerns. Aspectual requirements define those crosscutting concerns that have an impact across the software architecture. Aspect-oriented software development (AOSD), often referred to as aspect-oriented programming (AOP), is a relatively new software engineering paradigm that provides a process and methodological approach for defining, specifying, designing, and constructing aspects.
- ✓ *AOCE uses a concept of horizontal slices through vertically decomposed software components, called "aspects," to characterize cross-cutting functional and non-functional properties of components. Common, systemic aspects include user interfaces, collaborative work, distribution, persistency, memory management, transaction processing, security, integrity and so on.*

MODULE-1

1. Define Software Engineering. Explain the characteristics of software.
2. Explain the unique nature of web applications?
3. Define software engineering and explain the layers of software engineering?
4. Define a software process and explain the generic software process framework?
5. What are the various Umbrella activities of a software process?
6. Explain the essence of software engineering practice?
7. What is a software myth and explain various types of myths?
8. Which process model leads to software reuse? Justify
9. Explain the following software process models?
    a. Waterfall model
    b. V model
    c. Incremental model
    d. Evolutionary models: prototyping model, spiral model, concurrent model
10. Other models
    a. Component based model
    b. Formal Methods Model
    c. Aspect Oriented Software Development

VTUSYNC.IN